



Python Course 2025/26/2

Lecture 7: Advanced OOP, Testing, and Debugging



Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- Pythonic class design with special methods and method types
- Multiple inheritance, MRO, ABCs, and where Protocols fit
- Object identity, mutation, argument passing, and CPython memory behavior
- Debugging with tracebacks, breakpoints, pytest, and TDD
- Architecture patterns that keep testing fast and practical

Learning objective

Move from writing classes that merely work to writing classes that are easier to inspect, reason about, test, and extend.

By the end, you can

Choose method and abstraction patterns intentionally, explain Python's object model and argument behavior, debug with evidence, and run a modern quality loop for safer refactors.

Concepts

Code reading

Debugging

Testing

This lecture turns the to-do app from a small program into a maintainable codebase.



Advanced Class Design

Make objects feel natural to use and easier to inspect

Special Methods: `__str__` VS `__repr__`

Two audiences, one object

- `__str__` is for a human-friendly display.
- `__repr__` is for debugging and precise inspection.
- Good `repr` values reduce guesswork in logs, shells, and tracebacks.

Fun Python fact

If `__str__` is missing, `print(obj)` falls back to `__repr__`.

```
class Task:
    def __repr__(self):
        return (
            f"Task(description='{self.description}', "
            f"completed={self.completed})"
        )

    def __str__(self):
        status = "[x]" if self.completed else "[ ]"
        return f"{status} {self.description}"
```

The same object can expose a neat user view and a richer developer view.



Same Object, Two Different Displays

```
task = Task("Buy groceries", completed=True)

print(task)
print(repr(task))
```

Expected output

```
[x] Buy groceries
Task(description='Buy groceries',
completed=True)
```

When each one helps

- User-facing CLI output: `__str__`
- Debugging sessions and logs: `__repr__`
- Shell inspection: typing the variable name often shows `repr(obj)`

Rule of thumb: always implement a useful `__repr__`; add `__str__` when users need a cleaner message.



Class Variables vs Instance Variables

```
class Task:
    task_count = 0

    def __init__(self, description):
        self.description = description
        self.completed = False
        Task.task_count += 1
```

Every task has its own description, but all tasks share one class-level counter.

How to tell them apart

- Instance variable: stored on one object via `self`
- Class variable: stored once on the class itself
- Instances can read class variables, but that does not make them instance-specific

Common mistake: using a mutable class variable for per-object data makes instances accidentally share state.



Class methods: `@classmethod`

```
class Task:
    def __init__(self, description, completed=False):
        self.description = description
        self.completed = completed

    @classmethod
    def from_line(cls, line):
        """Parse '[x] ... ' or '[ ] ... ' into a Task."""
        line = line.strip()
        if line.startswith("[x]"):
            return cls(line[3:].strip(), completed=True)
        if line.startswith("[ ]"):
            return cls(line[3:].strip(), completed=False)
        return cls(line, completed=False)
```

The first parameter is conventionally `cls` (the class object). Use `cls( ... )` so subclasses get the right type.

Why use them?

- **Alternative constructors** (factories): same class, different input shapes.
- Logic that needs the class (hooks, registries, defaults on the class) but not a particular instance yet.
- Call style: `Task.from_line("[x] Done")` reads clearly at the call site.

Contrast: an instance method uses `self` because it works with one object that already exists.

Static methods: `@staticmethod`

```
class Task:
    # ... __init__ and other methods ...

    @staticmethod
    def normalize_description(text):
        """Pure helper: collapse whitespace; no access to class or instance."""
        return " ".join(text.split())

# Call through the class or an instance – same effect
Task.normalize_description("  buy    milk  ")
t = Task("demo")
t.normalize_description("  too  many  spaces  ")
```

Python does not pass `self` or `cls`. The method is just a namespaced function living on the class.

When they fit

- Small utilities that conceptually belong with the type but need no instance or class state.
- Often a module-level function is enough; use `@staticmethod` when grouping under the class helps discovery (API surface).



Choose the Right Method Type

Instance

`self`

Use it when behavior reads or changes one object's state.

Class

`cls`

Use it for factory methods and class-wide behavior.

Static

No implicit first argument
Use it for utility logic tied to the class concept but not to class or instance state.

Kind	First argument	Typical use
Instance	<code>self</code>	Per-object behavior and data
Class	<code>cls</code>	Factories, class-level setup
Static	(none)	Stateless helper on the namespace

Quick rule: state-aware logic belongs in instance or class methods. If a helper needs no state, consider a module-level function first; keep it static only when class-local discoverability helps.



Checkpoint: Advanced Class Design

Q1

If a class defines `__repr__` but not `__str__`, what does `print(obj)` use?

- A) It raises an error immediately
- B) It falls back to `__repr__`
- C) It always prints an empty string
- D) It ignores both methods and prints nothing useful

Show Answer

Q2

After creating two `Task` objects, which value should a shared class variable like `Task.task_count` hold?

- A) One separate count per instance
- B) 2, because the class-level counter is shared
- C) 0, because only instances can change it
- D) A random value based on instance creation order

Show Answer

Q3

Which statement is true about `@classmethod` and `@staticmethod`?

- A) Both pass `self` as the first argument
- B) `@classmethod` passes the class as the first argument; `@staticmethod` passes neither `self` nor `cls`
- C) `@staticmethod` passes `cls`; `@classmethod` does not
- D) You may define only one of the two per class

Show Answer



Inheritance and Abstraction

Reuse behavior carefully and define clear contracts



Multiple Inheritance: Mix Capabilities Carefully

```
class Phone:
    def make_call(self):
        print("Calling ... ")

class Camera:
    def take_picture(self):
        print("Snap!")

class Smartphone(Phone, Camera):
    pass
```

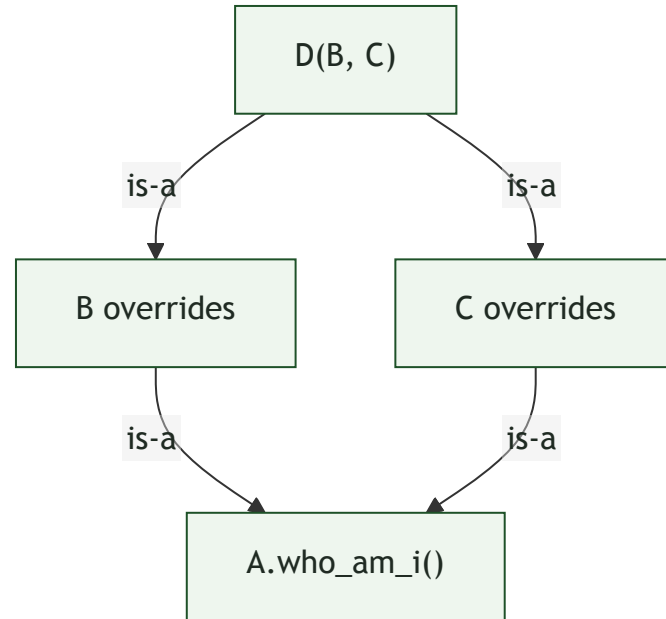
One child class can reuse behavior from more than one parent.

Why people use it

- To mix independent capabilities into one object
- To avoid duplicating helper behavior
- To express "is both X and Y" relationships

Risk: if two parents define the same method, the child needs a predictable rule for deciding which implementation runs.

The Diamond Problem



If `B` and `C` both override the same method from `A`, which one should `D` use? Python solves that with the MRO.



Method Resolution Order (MRO)

```
class D(B, C):  
    pass  
  
print([cls.__name__ for cls in D.__mro__])  
# ['D', 'B', 'C', 'A', 'object']
```

`super()` means "call the next method in this order", not simply "call my parent".

What to remember

- Lookup order is predictable and inspectable
- Parents listed first matter
- Cooperative multiple inheritance depends on each class using `super()` consistently

Fun Python fact

Python computes the MRO with `C3 linearization`, which preserves local left-to-right priority while keeping the order consistent across the hierarchy.



Abstract Base Classes (ABCs) Define Contracts

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

A subclass must implement every abstract method before you can instantiate it.

Why ABCs help

- They document expected behavior as a clear contract
- They fail fast at runtime when implementations are incomplete
- They make dependency injection and test doubles safer to use

In modern codebases, ABCs are often paired with type hints; `typing.Protocol` can complement them when structural typing is enough.



Real Example: Storage ABC in the To-Do App

```
class Storage(ABC):  
    @abstractmethod  
    def load(self) → list[dict[str, object]]:  
        raise NotImplementedError  
  
    @abstractmethod  
    def save(self, tasks: list[dict[str, object]]) → None:  
        raise NotImplementedError
```

Both `JsonFileStorage` and `MemoryStorage` promise the same interface contract.

Why this matters later

- The manager can work with file storage or memory storage
- Tests become faster because they can avoid real file I/O
- Adding a new backend does not require rewriting the manager

Bridge to testing: a clean interface is what makes dependency injection and isolated unit tests practical.



`typing.Protocol`: Structural (“Duck”) Typing

```
from typing import Protocol, runtime_checkable

class Drawable(Protocol):
    def draw(self) → None: ...

def render(obj: Drawable) → None:
    obj.draw()
```

A type checker treats any object with a matching `draw` signature as a `Drawable`, no need to inherit from the protocol class.

What Protocol is for

- Documents “I need these methods/attributes” for static analysis
- Fits third-party or legacy classes you cannot subclass
- Keeps APIs flexible: implement the shape, not the name on the inheritance line

Runtime: protocols are mostly for type checkers. Add `@runtime_checkable` if you need `isinstance()` checks against the protocol.

ABC vs `Protocol`



	Abstract Base Class (<code>ABC</code>)	<code>typing.Protocol</code>
Kind of contract	Nominal — types must subclass the ABC (or register) to be “officially” part of the family	Structural — if the object has the right methods/attributes, it matches
Who enforces it	Python at instantiation : missing abstract methods → <code>TypeError</code>	Primarily static type checkers (<code>mypy</code> , Pyright, ...); optional <code>isinstance</code> with <code>@runtime_checkable</code>
Typical use	Shared implementation, strict hierarchy, “this is a <code>Shape</code> ”	Callbacks, pluggable dependencies, “this behaves like something drawable”

They solve overlapping problems: ABCs lean on inheritance and runtime safety; protocols lean on shape and checker-time clarity. Many projects use both.



Checkpoint: Inheritance and Abstraction

Q4

In `class D(B, C)`, which parent wins first when both define the same method?

- A) `B`, because it appears first in the child definition
- B) `C`, because it is alphabetically later
- C) `A`, because the ancestor is always safer
- D) Both parent methods run in parallel

Show Answer

Q5

What does `super()` mean in cooperative multiple inheritance?

- A) Call the direct parent by hardcoded class name
- B) Call the next method in the MRO
- C) Skip every parent and call the child again
- D) Jump straight to `object`

Show Answer

Q6

What happens when you instantiate a subclass of `ABC` that still leaves an `@abstractmethod` unimplemented?

- A) Python raises `TypeError` at instantiation time
- B) Python raises `SyntaxError` when the subclass body is parsed
- C) Missing abstract methods silently use the parent's `pass` stubs
- D) It only fails if you run a static type checker

Show Answer



Python Under the Hood

Mental models for objects, references, and memory



Core Philosophy: Everything Is an Object

Values are objects

1 is an `int`.
"hello" is a `str`.
[1, 2, 3] is a `list`.

Functions are objects

```
def foo():  
    pass  
  
print(type(foo))  
# function
```

Classes are objects

```
class Task:  
    pass  
  
print(type(Task))  
# type
```

Fun Python fact

Functions and classes are first-class objects, so you can store them in variables, pass them to functions, and inspect them at runtime.

Identity, Type, and Value

Three things every object has

- **Identity:** which object it is
- **Type:** what operations it supports
- **Value:** the current data it holds

Use `id()` for identity and `type()` for type.

```
name = "Ada"
numbers = [1, 2, 3]

print(type(name))
print(type(numbers))
print(id(numbers))
```

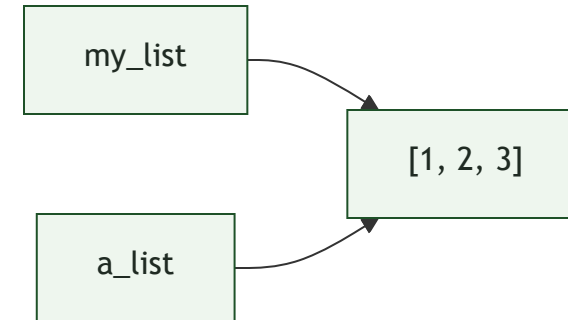
Mutation reminder: immutable objects change by creating a new object; mutable objects can change in place.



How Python Passes Arguments

The right mental model

- Python binds function parameters to existing objects at call time.
- Mutating a shared mutable object is visible to the caller.
- Rebinding a local name points that name to a different object.



The names are different, but at call time they can point to the same mutable object.

Mutation vs Rebinding



Immutable example

```
def add_one_to_num(a_num):  
    a_num = a_num + 1  
  
my_num = 10  
add_one_to_num(my_num)  
print(my_num)  # 10
```

Rebinding `a_num` does not change the caller's original integer.

Mutable example

```
def add_to_list(a_list):  
    a_list.append(4)  
  
my_list = [1, 2, 3]  
add_to_list(my_list)  
print(my_list)  # [1, 2, 3, 4]
```

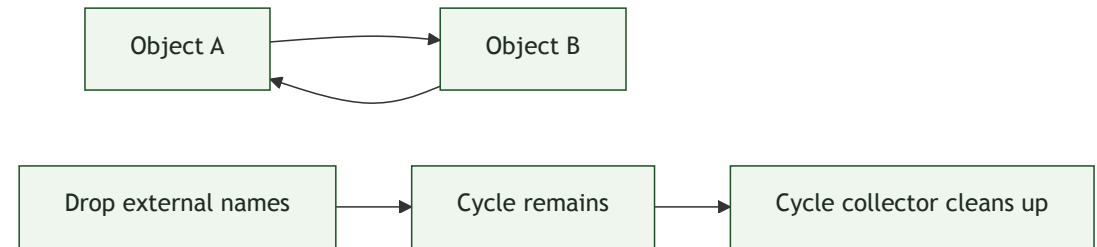
Mutating the shared list changes what the caller sees too.

Key distinction: mutating an object and rebinding a local variable are not the same operation.

Automatic Memory Management (CPython Focus)

Reference counting

- Create `x = [1, 2]` -> one live reference
- Assign `y = x` -> another reference to the same object
- Delete or rebind names -> reference count can decrease
- When references drop to zero, CPython usually reclaims the object quickly



Fun Python fact

CPython combines refcounting with a cyclic garbage collector. Finalization timing can differ across runtimes, so use context managers for critical resource cleanup instead of relying on `__del__`.



Checkpoint: Python Under the Hood

Q7

After calling a function that runs `a_list.append(4)`, what does the caller see?

- A) The original list unchanged
- B) The list with 4 added
- C) A traceback every time
- D) A new tuple with 4 added

Show Answer

Q8

What is the type of a user-defined class object like `Task`?

- A) function
- B) object
- C) type
- D) module

Show Answer



Debugging, Testing, and TDD

Make invisible bugs visible before they escape



Debugging Is a Method, Not a Panic Button

What not to do

- Panic and start changing random lines
- Guess without gathering evidence
- Depend only on endless `print()` calls

What to do instead

- Reproduce the bug reliably
- Shrink to a minimal reproducible case
- Pause execution near suspicious state with breakpoints
- Inspect values and step through the exact code path
- Confirm the fix by rerunning

A debugger turns "I think" into "I can see the actual state now." Use IDE breakpoints or `breakpoint()` when you need a quick in-code pause.



Read Tracebacks from the Bottom Up

```
Traceback (most recent call last):
```

```
  File "main.py", line 12, in <module>  
    run()
```

```
  File "app.py", line 8, in run  
    total = add("2", 3)
```

```
TypeError: can only concatenate str (not "int") to str
```

How to parse it

- **Bottom line:** what failed
- **Lines above:** where the failing call came from
- Use the stack to reconstruct the path to the bug
- Python 3.11+ often highlights the exact failing expression

If you only read the first line, you usually miss the actual cause. Start at the bottom, then walk upward through frames.



Bug Hunt: String Key vs Integer Key

```
TASKS_DB = {  
    1: {"desc": "Buy milk"},  
    2: {"desc": "Do laundry"},  
}  
  
def find_task_by_id(task_id):  
    if task_id in TASKS_DB:  
        return TASKS_DB[task_id]  
    return None
```

The user input is a string like "2", but the dictionary keys are integers.

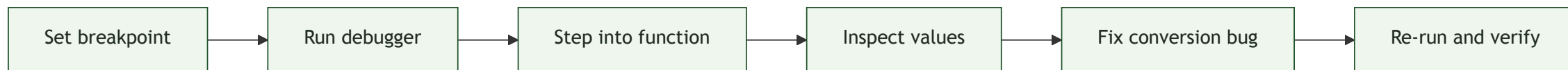
Why the bug happens

- "2" and 2 differ in both type and value semantics
- The membership test therefore returns False
- Fix near the boundary: convert and validate user input before lookup

Type mismatches are easier to see when you inspect runtime values. A robust fix is `int(task_id)` with `ValueError` handling.



Debugger Workflow in VS Code



Breakpoints are valuable because they freeze the program exactly at the moment where your mental model and reality start to differ.

Why Do We Write Tests?

Tests give us

- Confidence to refactor without fear
- Early bug detection before users hit the bug
- Executable examples of expected behavior
- Pressure toward smaller, more modular code

Tests are not extra paperwork. They are a feedback tool that makes changes safer.

Good design side effect

If something is painful to test, that often reveals a coupling or responsibility problem in the design.



Pytest: Low Ceremony, Fast Feedback

```
from tasks import add

def test_add_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2
```

Simple file names and function names are enough for pytest discovery.

Why pytest stays the default

- Plain `assert` statements
- No test class boilerplate required
- Helpful failure messages
- Course workflow: `uv add --dev pytest`, then `uv run pytest -q`

Fun Python fact

`assert` statements can disappear under `python -O`, so use them for tests and invariants, not for user-facing validation logic.



Fixtures and `tmp_path` Keep Tests Clean

```
def test_save_tasks_to_file(tmp_path):  
    test_file = tmp_path / "test_tasks.json"  
    storage = JsonFileStorage(str(test_file))  
    storage.save([{"description": "Learn", "completed": False}])  
    assert test_file.exists()
```

Each test gets its own temporary directory and fresh file path.

Why `tmp_path` is so useful

- Isolation: tests do not pollute real files
- Cleanup: pytest removes temporary directories automatically
- Portability: works across macOS, Linux, and Windows
- Modern style: `tmp_path` is a `pathlib.Path` object

Defining fixtures: `@pytest.fixture`

```
import pytest

@pytest.fixture
def empty_task_list():
    return []

def test_len_empty(empty_task_list):
    assert len(empty_task_list) == 0
```

The parameter name must match the fixture function name; pytest wires it in before the test runs.

What fixtures are for

- Reuse setup that many tests need (data, clients, config)
- Keep each test short: arrange complex state once, inject it
- One fixture, one responsibility—compose small fixtures instead of one giant setup

Teardown

Use `yield` in the fixture body: code after `yield` runs after the test, even if the test fails.



`conftest.py` and fixture scope

```
# tests/conftest.py – discovered automatically
import pytest

@pytest.fixture(scope="module")
def shared_config():
    return {"debug": True}
```

Place shared fixtures in `conftest.py` under `tests/` (or a package) so every test module in that tree can request them by name.

Scopes (default: `function`)

Scope	Created
<code>function</code>	Once per test (safest default)
<code>class</code> / <code>module</code>	Shared across tests in that class or file
<code>session</code>	Once for the whole run, watch mutable shared state

Good default practice

- Start with `scope="function"` ; widen only when setup is expensive and shared state is clearly safe.
- Wider scopes + mutable objects: one test can affect another via the same fixture instance.
- Pytest's fixture model scales across scopes, but explicit, easy-to-read fixture wiring is usually better than clever scope tricks.



Fixture best practices

Do

- Name fixtures after the resource they provide (`db_connection`, not `setup`)
- Prefer `tmp_path` for files; use `monkeypatch` for env vars and small fakes
- Build bigger setups from smaller fixtures (pytest resolves the dependency graph)
- Use `params` on a fixture when every test needs the same matrix of inputs

Avoid

- `autouse=True` unless every test in the file truly needs it, hidden dependencies are hard to read
- Session-scoped fixtures that mutate global state without teardown
- Fixtures that assert or “test” things; keep assertions inside tests only
- Over-nesting: if setup is unreadable, extract a helper or factory function

Rule of thumb

A reader should see every dependency in the test signature. Explicit fixture parameters beat magic global setup.



What a Good Test Looks Like

Arrange

Create the objects, input data, and environment for the test.

Act

Run the one behavior you actually want to verify.

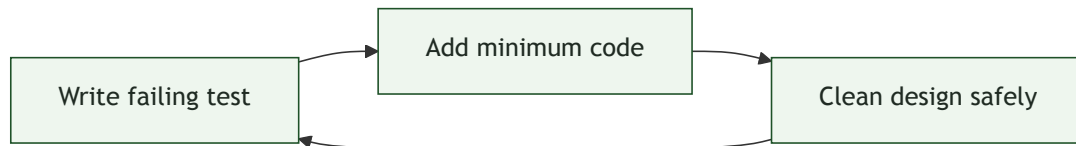
Assert

Check observable outcomes with clear expectations.

Quality checklist: keep tests deterministic (no hidden time/network dependencies), focus each test on one behavior, and use parametrization when only inputs vary.



TDD: Red - Green - Refactor



Why teams still use it

- Forces you to define behavior before implementation
- Builds a safety net before refactoring
- Encourages smaller steps and tighter feedback loops
- Works best for design-heavy or bug-prone behavior, not every trivial line

TDD is a tool, not a religion. Use it deliberately where fast feedback improves design decisions.



TDD Example: Add Priority to Task

```
def test_task_creation_with_priority():  
    task = Task("Buy milk", priority=1)  
    assert task.description == "Buy milk"  
    assert task.priority == 1
```

The first run fails because the feature does not exist yet.

Then we improve it in stages

- **Red:** failing test proves the gap
- **Green:** add the minimal constructor change
- **Refactor:** set default priority and validate allowed range

Notice how the test turns a vague idea like "support priority" into a precise, checkable contract.



Checkpoint: Debugging, Testing, and TDD

Q9

When you read a traceback, where should you usually start?

- A) The bottom line with the exception and message
- B) The first blank line you can find
- C) Somewhere in the middle for suspense
- D) The very first frame only

Show Answer

Q10

Why is `tmp_path` such a good fit for file-based tests?

- A) It gives each test a clean temporary directory
- B) It avoids collisions with real project files
- C) It disables all file errors forever
- D) It makes every OS use the same path format

Show Answer



Project-Level Craftsmanship

Architecture choices that make real testing possible



Why Better Architecture Helps Testing

Before refactoring

- Task logic and file I/O lived too close together
- Side effects leaked into places that should stay pure
- Changing storage details risked breaking unrelated behavior

After refactoring

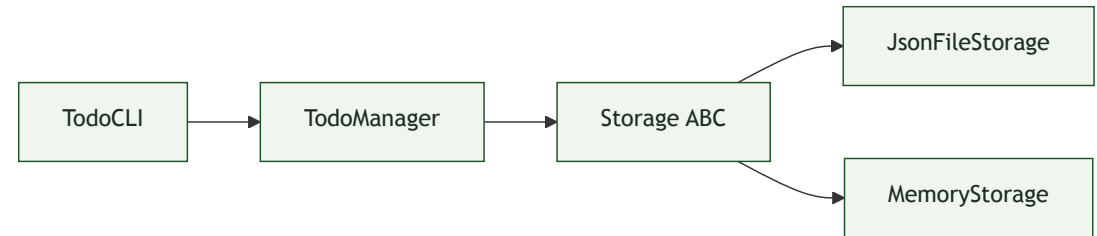
- `TodoManager` focuses on task behavior
- Storage implementations own persistence side effects
- Alternative backends become easy to swap in for tests

Good architecture is not abstract decoration. It directly lowers the cost of testing and change.

Storage, Manager, and CLI Fit Together

```
class TodoManager:
    def __init__(self, storage: Storage):
        self.storage = storage
        self.tasks = []
        self._load_tasks()
```

The manager receives a storage dependency instead of creating one internally.



Dependency injection lets the same manager work with production storage and test-friendly storage.



Testing File Storage Safely

```
def test_save_tasks_to_file(tmp_path):  
    test_file = tmp_path / "test_tasks.json"  
    storage = JsonFileStorage(str(test_file))  
    storage.save([  
        {"description": "Learn Python", "completed": False}  
    ])  
    assert test_file.exists()
```

What this test suite covers

- Basic save/load behavior
- Non-existent files
- Round-trip integrity
- Corrupted JSON and bad data shapes
- Formatting and independence of storage instances
- A foundation for parametrized edge-case tests



Live Coding and Next Iterations

Iteration 4 focus

- Create the `Storage` ABC
- Extract `JsonFileStorage`
- Add `MemoryStorage`
- Inject storage into the manager

Iteration 5 focus

- Write TDD-style tests for task rename behavior
- Write storage tests with `tmp_path`
- Use tests to protect further refactoring

Iteration 6 quality loop

- `uv sync`
- `uv run ruff check .`
- `uv run pyright`
- `uv run pytest -q`

The app is now big enough that quality tools are part of daily development, not a final cleanup step.



Lecture 7: Big Picture

Pythonic Classes

- Use special methods to improve readability
- Separate instance state from class-wide state
- Choose the right method type intentionally

Mental Models

- Everything is an object
- Arguments pass object references, not magical copies
- Memory management mixes refcounts and cycle cleanup

Craftsmanship

- Use debuggers to inspect evidence
- Keep tests deterministic and fast
- Automate lint, type, and test checks in CI

Today's core message: good OOP design, clear mental models, and disciplined testing reinforce each other. Better structure makes bugs easier to find and safer to prevent.



Thank You for Your Attention!

*"Programs must be written for people to read, and only
incidentally for machines to execute."*

Carry the debugger, the tests, and the design mindset into the next version of
your app.